Lista 8: Drzewa Decyzyjne

Uczenie Maszynowe – Uniwersytet Wrocławski

Semestr zimowy 2025/2026

Uwaga: Rozwiązania powinny zawierać kod w Pythonie oraz krótkie komentarze wyjaśniające kluczowe decyzje implementacyjne. Celem tej listy jest **odkrywanie** metod przez eksperymentowanie – staraj się najpierw zrozumieć intuicję, zanim sprawdzisz oficjalne implementacje w sklearn.

Zadanie 1: WroBank – Stabilizacja Niestabilnych Klasyfikatorów (3 pkt)

Scenariusz: Firma "WroBank" buduje system oceny ryzyka kredytowego. Pojedyncze drzewo decyzyjne daje różne wyniki przy każdym treningu (problem niestabilności). Klient narzeka: "Wczoraj model powiedział TAK, dziś mówi NIE dla tego samego wniosku!" Twój szef pyta: "Jak możemy uzyskać stabilne predykcje bez rezygnowania z drzew?" Twoim zadaniem jest odkrycie metody stabilizacji.

Dane: Użyj syntetycznych danych kredytowych generowanych przez `make_classification` z 1000 próbek i 10 cech.

Polecenia:

(a) **Niestabilność pojedynczego drzewa:**

- Wytrenuj 10 drzew decyzyjnych na tych samych danych (różne `random_state`: 0, 1, ..., 9)
- Dla każdego punktu testowego oblicz wariancję predykcji prawdopodobieństw między drzewami
- Zwizualizuj rozkład wariancji – ile punktów ma wysoką niestabilność?
- Narysuj granice decyzyjne kilku drzew na 2D (użyj pierwszych dwóch cech)

(b) **Agregacja predykcji (Bootstrap Aggregating):**

- Zaimplementuj losowanie bootstrap: n próbek ze zwracaniem z n oryginalnych
- Wytrenuj 50 drzew, każde na innej próbce bootstrap
- Dla klasyfikacji: zastosuj głosowanie większościowe
- Oblicz i porównaj wariancję predykcji z punktem (a) – o ile spadła?

(c) **Losowe podzbiory cech:**

- Zmodyfikuj procedurę: przy każdym podziale losuj $\sqrt{p}$ cech (gdzie p = liczba cech)
- Zaimplementuj funkcję `build_randomized_tree()` używając parametru `max_features='sqrt'`

- Porównaj drzewa z i bez losowania cech – które są bardziej "różnorodne"?
- Zbadaj korelację między predykcjami drzew w obu wariantach (Pearson na prawdopodob.)

(d) **Analiza końcowa:**

- Zbadaj wpływ liczby drzew: 10, 50, 100, 200, 500 – gdzie następuje "nasycenie"?
- Zaimplementuj OOB (Out-of-Bag) error: oceń model bez zbioru walidacyjnego
- Porównaj wyniki z `sklearn.ensemble.RandomForestClassifier` – czy są zgodne?
- Wyjaśnij matematycznie, dlaczego ta metoda redukuje wariancję (wzór na wariancję średniej)

Kod startowy:

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from collections import Counter
import matplotlib.pyplot as plt

def generate_credit_data(n_samples=1000, n_features=10, random_state=42)
    :
    """Generuje syntetyczne dane kredytowe."""
    X, y = make_classification(
        n_samples=n_samples, n_features=n_features,
        n_informative=6, n_redundant=2, n_clusters_per_class=2,
        random_state=random_state
    )
    return X, y

def bootstrap_sample(X, y, random_state=None):
    """Losuje probke bootstrap (n probek ze zwracaniem z n)."""
    rng = np.random.RandomState(random_state)
    n = len(X)
    indices = rng.choice(n, size=n, replace=True)
    oob_indices = list(set(range(n)) - set(indices))
    return X[indices], y[indices], oob_indices

def train_single_trees(X_train, y_train, n_trees=10):
    """Trenuje n pojedynczych drzew z roznymi seedami."""
    trees = []
    for seed in range(n_trees):
        tree = DecisionTreeClassifier(random_state=seed)
        tree.fit(X_train, y_train)
        trees.append(tree)
    return trees

def calculate_prediction_variance(trees, X):
    """Oblicza wariancje predykcji miedzy drzewami."""
    # TODO: Zbierz predict_proba ze wszystkich drzew
    # TODO: Oblicz wariancje dla kazdego punktu
    pass

def ensemble_predict(trees, X):
    """Agreguje predykcje wielu drzew przez glosowanie wiekszosciowe."""
```

```
    # TODO: Zbierz predykcje ze wszystkich drzew
    # TODO: Dla kazdego punktu zwroc najczesciej wystepujaca klase
    pass

def calculate_oob_error(X, y, trees, oob_indices_list):
    """Oblicza OOB error bez zbioru walidacyjnego."""
    # TODO: Dla kazdego punktu uzyj tylko drzew, ktore go nie widzialy
    # TODO: Zaglosuj i porownaj z prawdziwa etykieta
    pass
```

Zadanie 2: WroSecure – Wykrywanie Intruzów przez Izolację (4 pkt)

Scenariusz: Firma "WroSecure" monitoruje ruch sieciowy w poszukiwaniu cyberataków. Problem: anomalie są rzadkie (1-5% ruchu), różnorodne i trudne do zdefiniowania. Klasyczne metody (etykietowanie, reguły) nie działają. Twój kolega rzucił prowokacyjną myśl: "Anomalie są *dziwne*, więc powinno być łatwo je ODIZOLOWAĆ od reszty." Twoim zadaniem jest zbadanie tej intuicji.

Dane: Syntetyczne dane ruchu sieciowego z normalnymi wzorcami i anomaliami (DDoS, port scan).

Polecenia:

(a) **Losowe drzewa izolujące:**

- Zbuduj drzewo z całkowicie losowymi podziałami: losowa cecha, losowy próg w zakresie [min, max]
- **UWAGA:** Nie używaj etykiet! To metoda nienadzorowana
- Każdemu punktowi przypisz "głębokość izolacji" (path length do liścia)
- Zwizualizuj histogram głębokości: normalne vs anomalie – co widzisz?

(b) **Intuicja geometryczna:**

- Wygeneruj dane 2D: normalny klaster + kilka anomalii na brzegach
- Narysuj kolejne podziały drzewa na wykresie krok po kroku
- Pokaż, że anomalie są izolowane po 2-3 podziałach, normalne po 8-10
- Wyjaśnij: "rzadkie punkty = mało sąsiadów = szybka izolacja"

(c) **Ensemble i anomaly score:**

- Zbuduj 100 losowych drzew izolujących (każde na podpróbce 256 punktów)
- Uśrednij path length $E[h(x)]$ dla każdego punktu
- Znormalizuj do anomaly score: $s(x, n) = 2^{-E[h(x)]/c(n)}$
- gdzie $c(n) = 2H(n-1) - 2(n-1)/n$ (oczekiwana głębokość BST dla n elementów)

(d) **Ewaluacja i porównanie:**

- Porównaj z LOF (Local Outlier Factor) i One-Class SVM
- Oblicz precision, recall, F1 na danych z etykietami (test set)
- Zbadaj wpływ: liczba drzew (50, 100, 200), max_samples (64, 128, 256)

- Porównaj z `sklearn.ensemble.IsolationForest` – czy implementacja zgodna?

Kod startowy:

```
import numpy as np
from scipy.special import digamma  # dla H(n) = digamma(n+1) - digamma
    (1)

def generate_network_data(n_normal=950, n_anomaly=50, random_state=42):
    """Generuje dane ruchu sieciowego z anomaliami."""
    np.random.seed(random_state)
    # Normalny ruch - 3 klastry operacyjne
    normal = np.vstack([
        np.random.randn(n_normal//3, 4) * 0.5 + [50, 100, 0.8, 30],
        np.random.randn(n_normal//3, 4) * 0.5 + [55, 110, 0.75, 35],
        np.random.randn(n_normal//3, 4) * 0.5 + [48, 95, 0.85, 28],
    ])
    # Anomalie - rozne wzorce atakow
    anomalies = np.vstack([
        np.random.randn(n_anomaly//2, 4) * 0.3 + [200, 500, 0.1, 5],   #
            DDoS
        np.random.randn(n_anomaly//2, 4) * 0.3 + [10, 10, 0.99, 100],  #
            Port scan
    ])
    X = np.vstack([normal, anomalies])
    y = np.array([0]*len(normal) + [1]*len(anomalies))
    return X, y

class IsolationTree:
    """Pojedyncze drzewo izolujace."""
    def __init__(self, max_depth=None):
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, current_depth=0):
        """Buduje drzewo z losowymi podzialami (bez etykiet!)."""
        n_samples, n_features = X.shape

        # Warunek stopu
        if n_samples <= 1 or (self.max_depth and current_depth >= self.
            max_depth):
            return {'type': 'leaf', 'size': n_samples, 'depth':
                current_depth}

        # TODO: Losuj ceche i prog
        # feature = np.random.randint(0, n_features)
        # min_val, max_val = X[:, feature].min(), X[:, feature].max()
        # threshold = np.random.uniform(min_val, max_val)

        # TODO: Podziel dane i rekurencyjnie buduj poddrzewa
        pass

    def path_length(self, x, node=None, current_depth=0):
        """Oblicza glebokosc izolacji punktu x."""
        # TODO: Rekurencyjnie przejdz drzewo az do liscia
        pass

def c_factor(n):
```

```
    """Oblicza oczekiwana glebokosc BST dla n elementow."""
    if n <= 1:
        return 0
    return 2 * (np.log(n - 1) + 0.5772156649) - 2 * (n - 1) / n

def anomaly_score(path_lengths, n_samples):
    """Oblicza znormalizowany anomaly score."""
    c = c_factor(n_samples)
    return 2 ** (-np.array(path_lengths) / c)
```

Zadanie 3: WroGeo – Geometria Granic Decyzyjnych (3 pkt)

Scenariusz: Firma "WroGeo" klasyfikuje typy terenu na podstawie danych satelitarnych. Zauważyli, że drzewa decyzyjne tworzą "kanciaste" mapy terenu zamiast płynnych granic. Klient pyta: "Dlaczego mój las wygląda jak pikselowa grafika z lat 90?" Twoim zadaniem jest zrozumienie ograniczeń geometrycznych drzew i ich konsekwencji.

Dane: Syntetyczne dane 2D: półksiężyce (`make_moons`), okręgi (`make_circles`), spirale.

Polecenia:

(a) **Wizualizacja granic decyzyjnych:**

- Wygeneruj dane 2D: `make_moons`, `make_circles`, spirale
- Wytrenuj drzewa o głębokości: 1, 3, 5, 10, None (bez ograniczeń)
- Narysuj granice decyzyjne dla każdej głębokości (meshgrid + contourf)
- Opisz obserwacje: jak głębokość wpływa na "gładkość"?

(b) **Dlaczego prostopadłe do osi?:**

- Zaimplementuj wizualizację budowy drzewa krok po kroku
- Pokaż, że każdy podział $x_i < t$ dzieli przestrzeń linią prostopadłą do osi i
- Narysuj "diagram podziałów" dla prostego drzewa (głębokość 3)
- Porównaj z SVM (krzywoliniowe granice z jądrem RBF) na tych samych danych

(c) **Ile podziałów potrzeba?:**

- Dla okręgu o promieniu r: ile podziałów potrzeba do aproksymacji z dokładnością ϵ?
- Zaimplementuj funkcję `count_splits_for_circle(radius, tolerance)`
- Wykres: tolerance vs liczba liści – jaki wzrost? (liniowy, kwadratowy, wykładniczy?)
- Wnioski dla danych rzeczywistych o "okrągłych" klastrach

(d) **Oblique Decision Trees:**

- Drzewa z podziałami ukośnymi: $a_1x_1 + a_2x_2 < t$
- Zaimplementuj prosty wariant (przeszukaj kilka kombinacji współczynników)
- Porównaj granice decyzyjne z drzewem axis-parallel
- Dlaczego sklearn tego domyślnie nie implementuje? (Wskazówka: złożoność obliczeniowa)

Kod startowy:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons, make_circles
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

def make_spirals(n_samples=500, noise=0.1, random_state=42):
    """Generuje dane w ksztalcie spiral."""
    np.random.seed(random_state)
    n = n_samples // 2
    theta = np.linspace(0, 4*np.pi, n)
    r = theta / (4*np.pi)
    x1 = r * np.cos(theta) + noise * np.random.randn(n)
    y1 = r * np.sin(theta) + noise * np.random.randn(n)
    x2 = -r * np.cos(theta) + noise * np.random.randn(n)
    y2 = -r * np.sin(theta) + noise * np.random.randn(n)
    X = np.vstack([np.column_stack([x1, y1]), np.column_stack([x2, y2])
        ])
    y = np.array([0]*n + [1]*n)
    return X, y

def plot_decision_boundary(model, X, y, ax, title=""):
    """Rysuje granice decyzyjna modelu."""
    h = 0.02  # krok siatki
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdYlBu')
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap='RdYlBu', edgecolors='black')
    ax.set_title(title)

def visualize_tree_splits(tree, X, y, ax):
    """Wizualizuje kolejne podzialy drzewa."""
    # TODO: Rekurencyjnie narysuj linie podzialow
    # Uzyj tree.tree_.feature, tree.tree_.threshold
    pass

def count_splits_for_circle(radius, tolerance):
    """Liczy ile podzialow potrzeba do aproksymacji okregu."""
    # TODO: Symuluj drzewo aproksymujace okrag
    pass
```